

sync.Mutex.state и lockSlow()

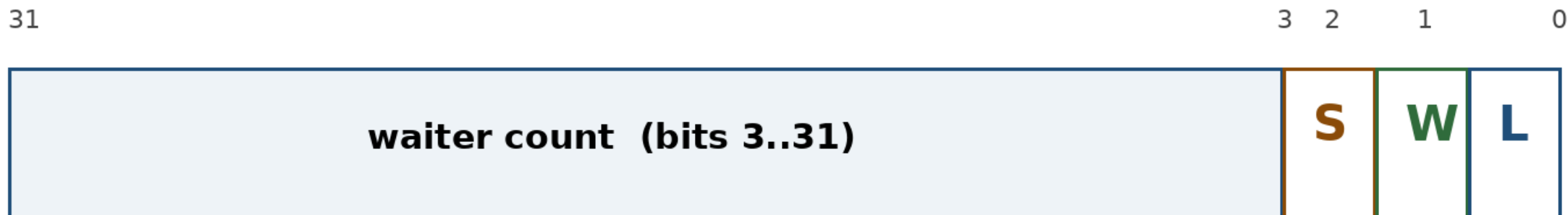
- Цель: понять Mutex как адаптивный runtime-протокол, а не как две функции Lock/Unlock.
- Путь: state bits → CAS fast path → spin → waiter accounting → semaphore → starvation handoff.
- Все конкретные биты и пороги — детали текущей реализации Go runtime.

Базовая структура Mutex

```
type Mutex struct {  
    state int32  
    sema  uint32  
}
```

Вся логика contention упакована в 32-битное state и runtime semaphore.

Mutex.state: 32 бита одной state machine



- L** lock занят обычным владельцем
- W** один активный contender уже назначен на пробуждение
- S** mutex в starvation mode; очередь получает приоритет
- waiters** количество sleeping/зарегистрированных waiters = $\text{state} \gg 3$

Константы state

```
const (  
    mutexLocked = 1 << iota  
    mutexWoken  
    mutexStarving  
    mutexWaiterShift = iota  
)
```

После трёх флагов остальные биты кодируют waiter count.

Как читать state

Примеры

- `state = 0`` → свободен
- `state = 1`` → Locked
- `state = 9`` → Locked + 1 waiter
- `state = 11`` → Locked + Woken + 1 waiter

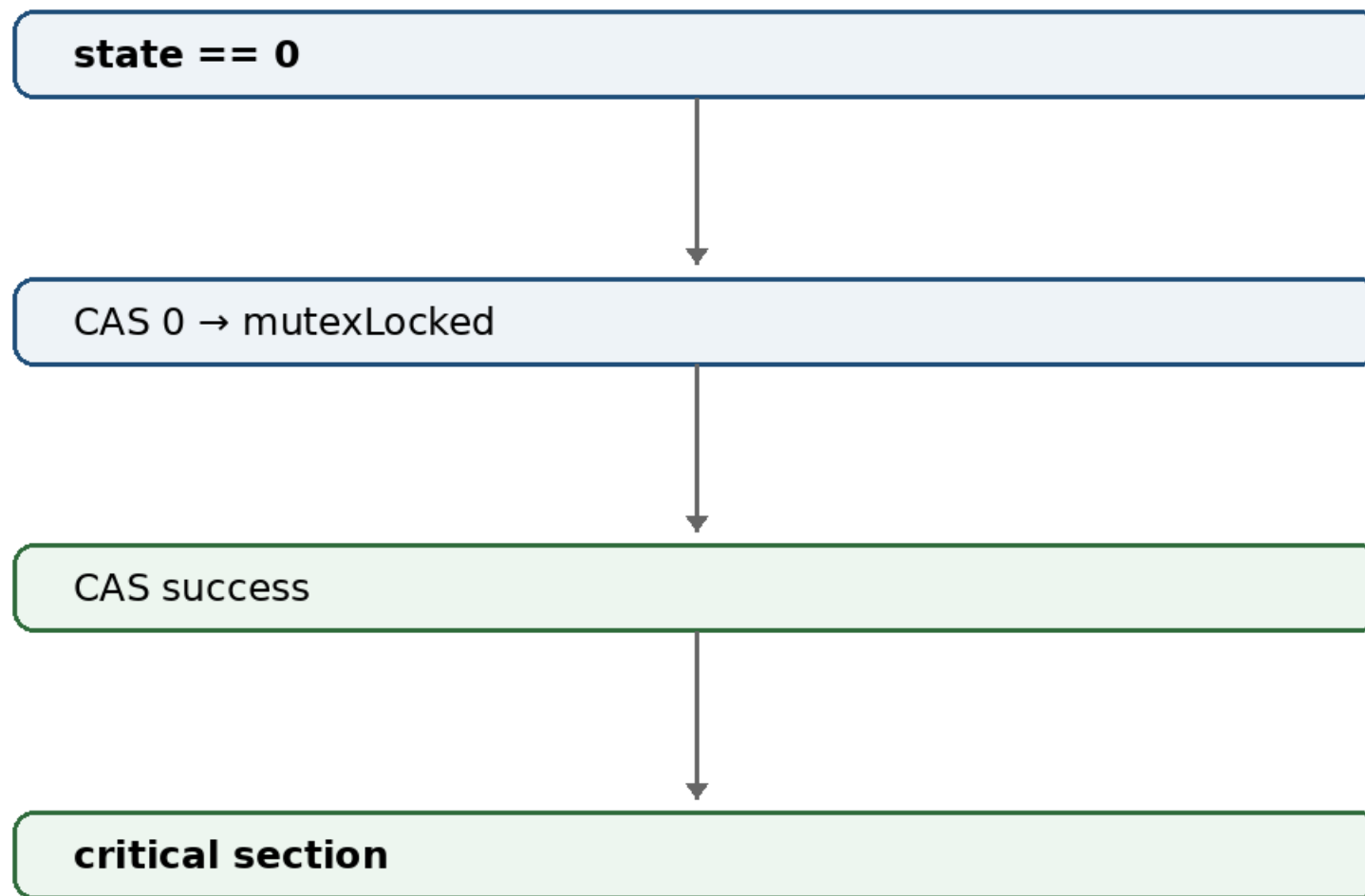
Формула

- `waiters = state >> 3`
- `+8`` означает ещё одного waiter
- младшие три бита — режим/координация

Зачем всё в одном `int32`?

- Связанные признаки изменяются одним CAS: `old state` → `new state`.
- Никакого отдельного `lock` для самого `Mutex.state` не требуется.
- Это компактная `state machine`: прочитал снимок → вычислил новый → попытался CAS → при конфликте пересчитал.

Lock(): common case должен быть почти бесплатным



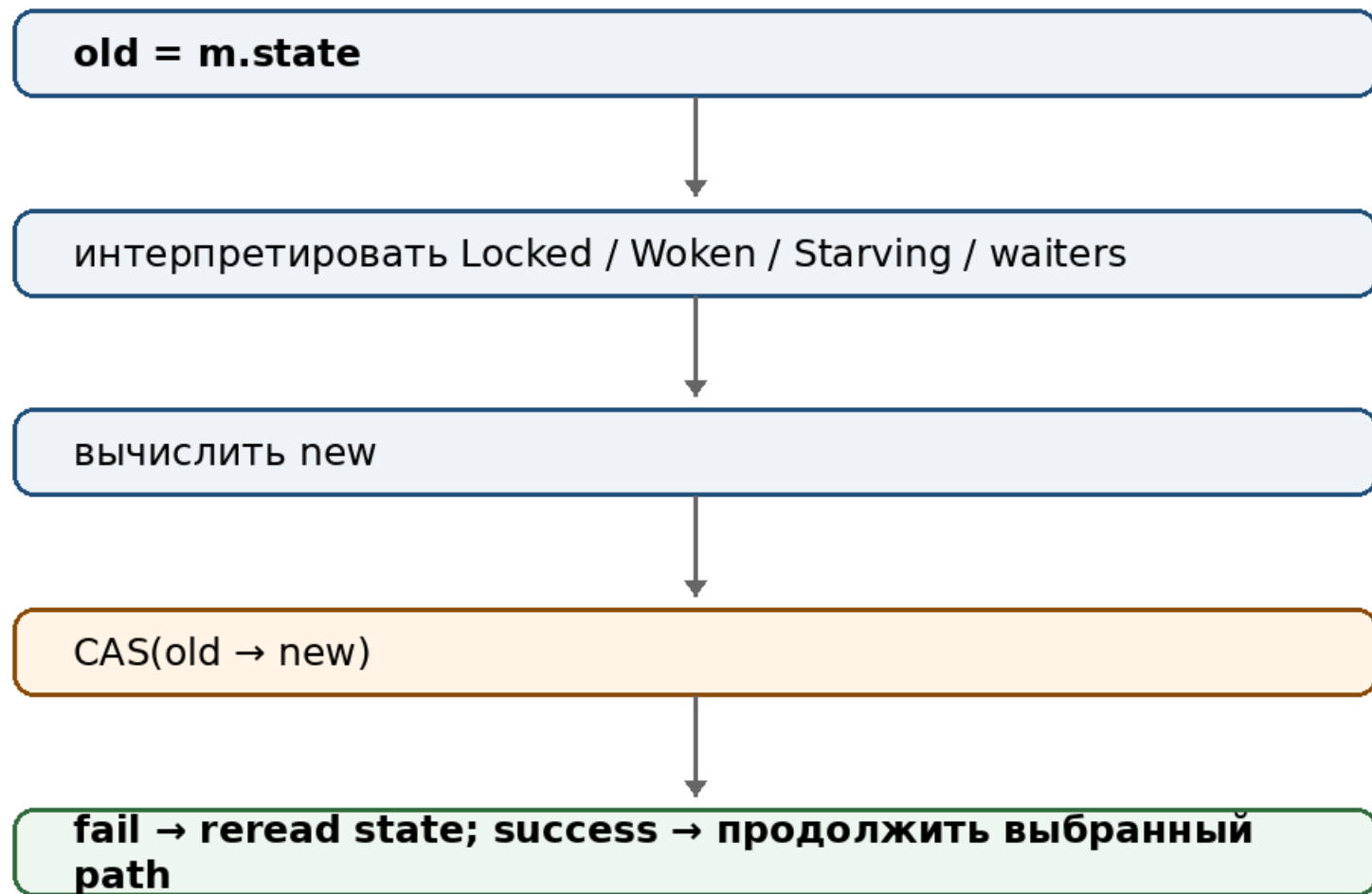
Uncontended path не идёт в scheduler, semaphore и sudog.

Fast path

```
if atomic.CompareAndSwapInt32(  
    &m.state,  
    0,  
    mutexLocked,  
) {  
    return  
}  
  
m.lockSlow()
```

Вся сложность вынесена из common path в lockSlow().

lockSlow(): центральная CAS state machine



Локальные переменные lockSlow()

- ``waitStartTime`` — когда началось реальное ожидание.
- ``starving`` — эта конкретная G считает ожидание слишком долгим.
- ``awoke`` — эта попытка Lock уже участвует как активный пробуждённый contender.
- ``iter`` — сколько раз мы пытались active spin.

mutexWoken: зачем отдельный bit?

- Mutex может иметь sleeping waiters и одновременно уже активного contender.
- `mutexWoken=1` говорит Unlock: кандидат уже есть, лишнего waiter будить не надо.
- Это уменьшает park → wake → schedule → проигрыш CAS → park churn.

Без mutexWoken: thundering herd



Одно освобождение ресурса не требует пробуждать толпу.

awoke vs mutexWoken

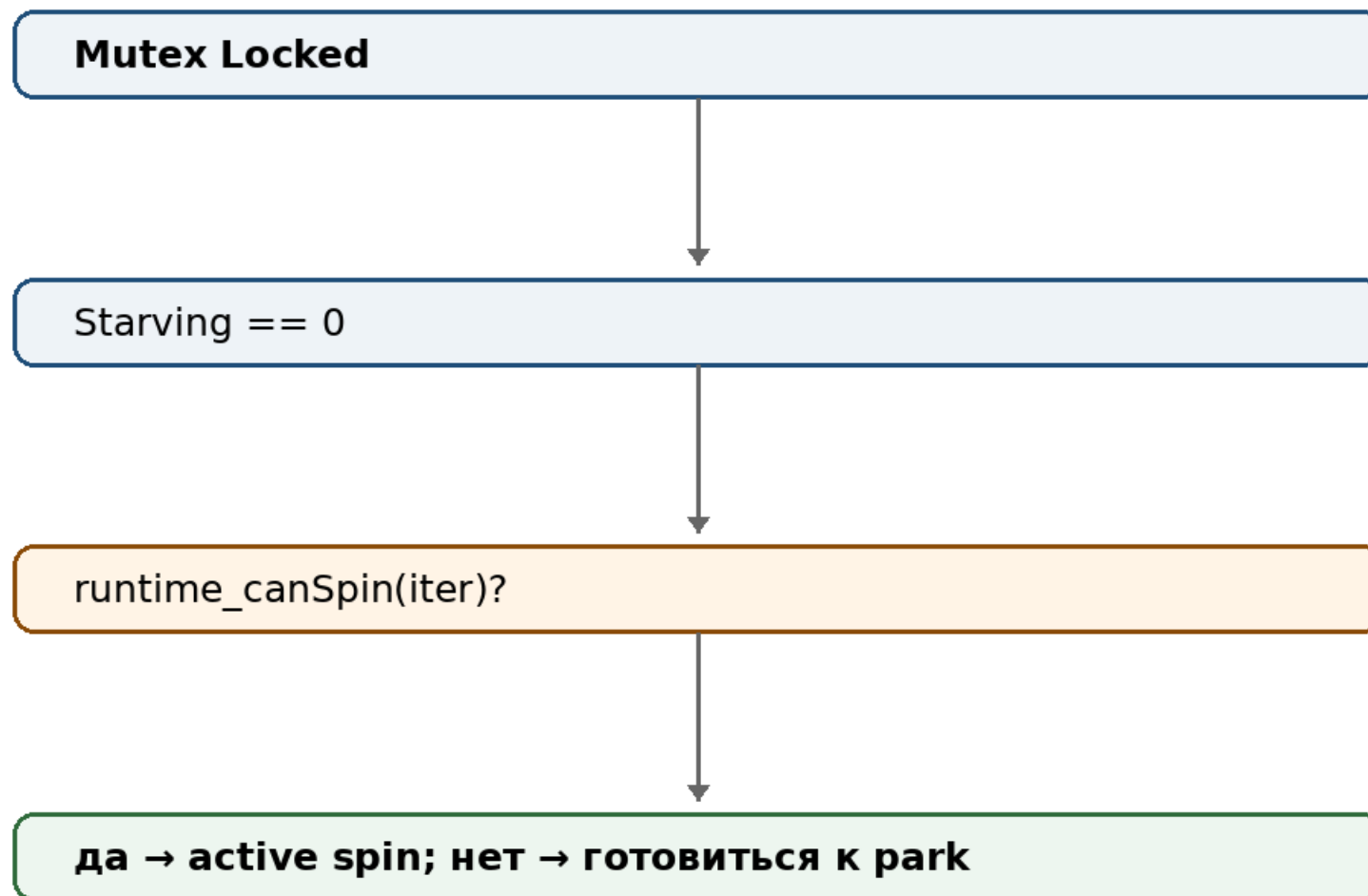
awoke

- локальная переменная одного Lock()
- значит: именно эта G уже активный кандидат
- позже эта G должна корректно снять Woken bit

mutexWoken

- глобальный bit в m.state
- координирует Unlock и contenders
- предотвращает лишнее пробуждение

Первый выбор lockSlow: можно ли spin?



Что такое active spinning

- G остаётся RUNNING и несколько раз ждёт освобождения lock, не отдавая P.
- Цена: M/P/CPU заняты почти бесполезной работой.
- Выгода: для очень короткой critical section park/unpark может быть дороже самого ожидания.

Когда spin разумен

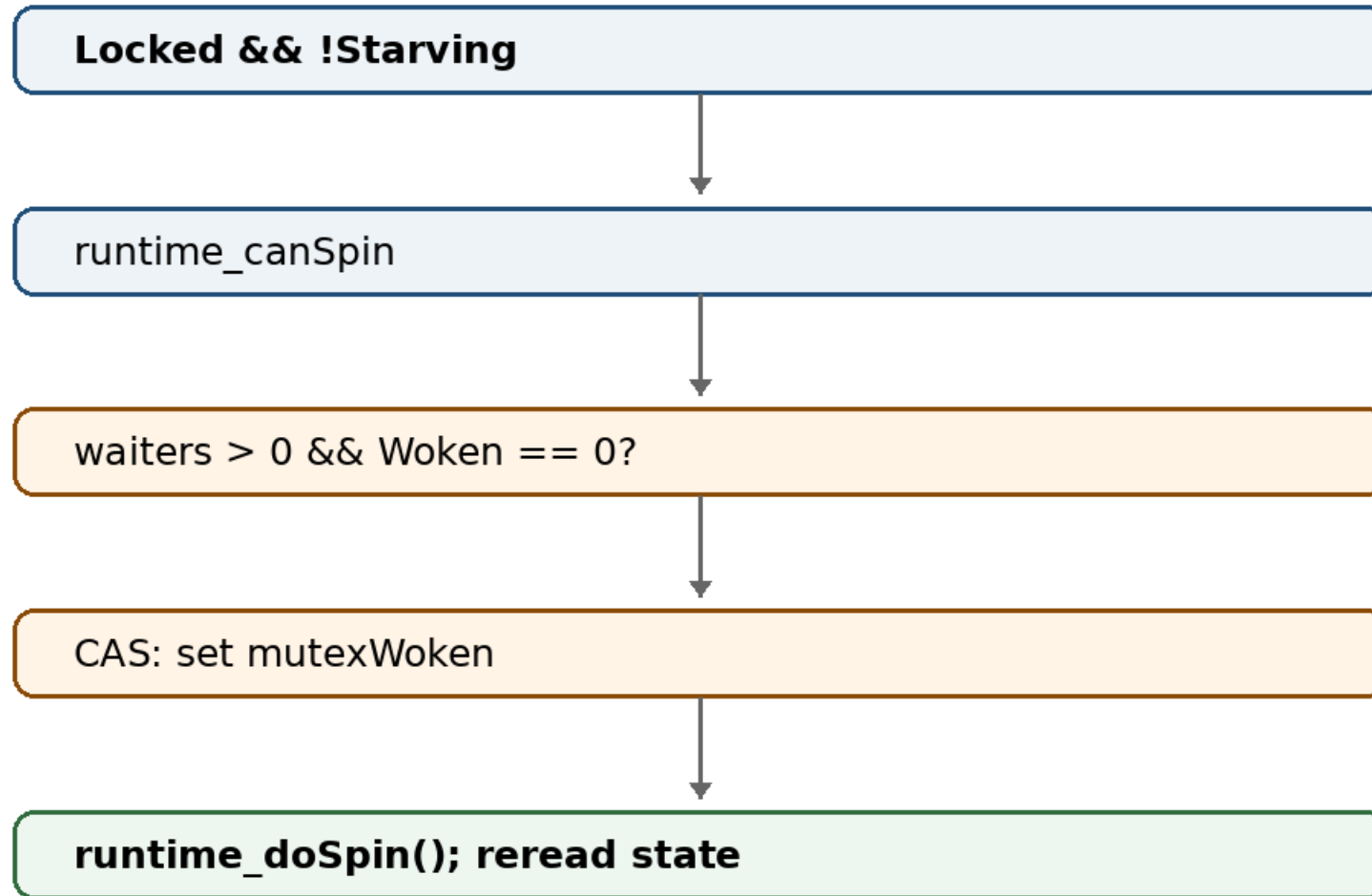
Хороший случай

- несколько CPU/P
- другая G реально может освободить lock
- локальная run queue текущего P пуста
- ожидание ожидается коротким

Плохой случай

- GOMAXPROCS=1
- есть runnable work
- critical section миллисекунды
- starvation mode уже активен

Spin path и mutexWoken



После spin: строим new state

- Если normal mode — текущая G может запросить ``mutexLocked``.
- Если mutex занят или starving — ``waiter count++`` (``+ 1 << 3``).
- Если G ждала слишком долго — готовим ``mutexStarving``.
- Если ``awoke=true`` — текущая попытка должна снять глобальный ``mutexWoken``.

Waiter count — это просто старшие биты

```
new += 1 << mutexWaiterShift  
  
// mutexWaiterShift == 3  
// => +8 к state = +1 waiter
```

Например: Locked + 2 waiters = 1 + 16 = 17.

Почему waiter++ происходит до sleep



Иначе Unlock может увидеть waiters=0 и потерять wakeup.

Starvation: когда throughput начинает ломать fairness

- В normal mode awakened waiter ещё конкурирует с newly arriving goroutines.
- Новая G уже RUNNING и может выиграть CAS раньше старого waiter — barging.
- Если один waiter проигрывает слишком долго, runtime переключает mutex в starvation mode.
- Текущий implementation threshold — порядка 1 ms; это не API guarantee.

Normal mode vs starvation mode

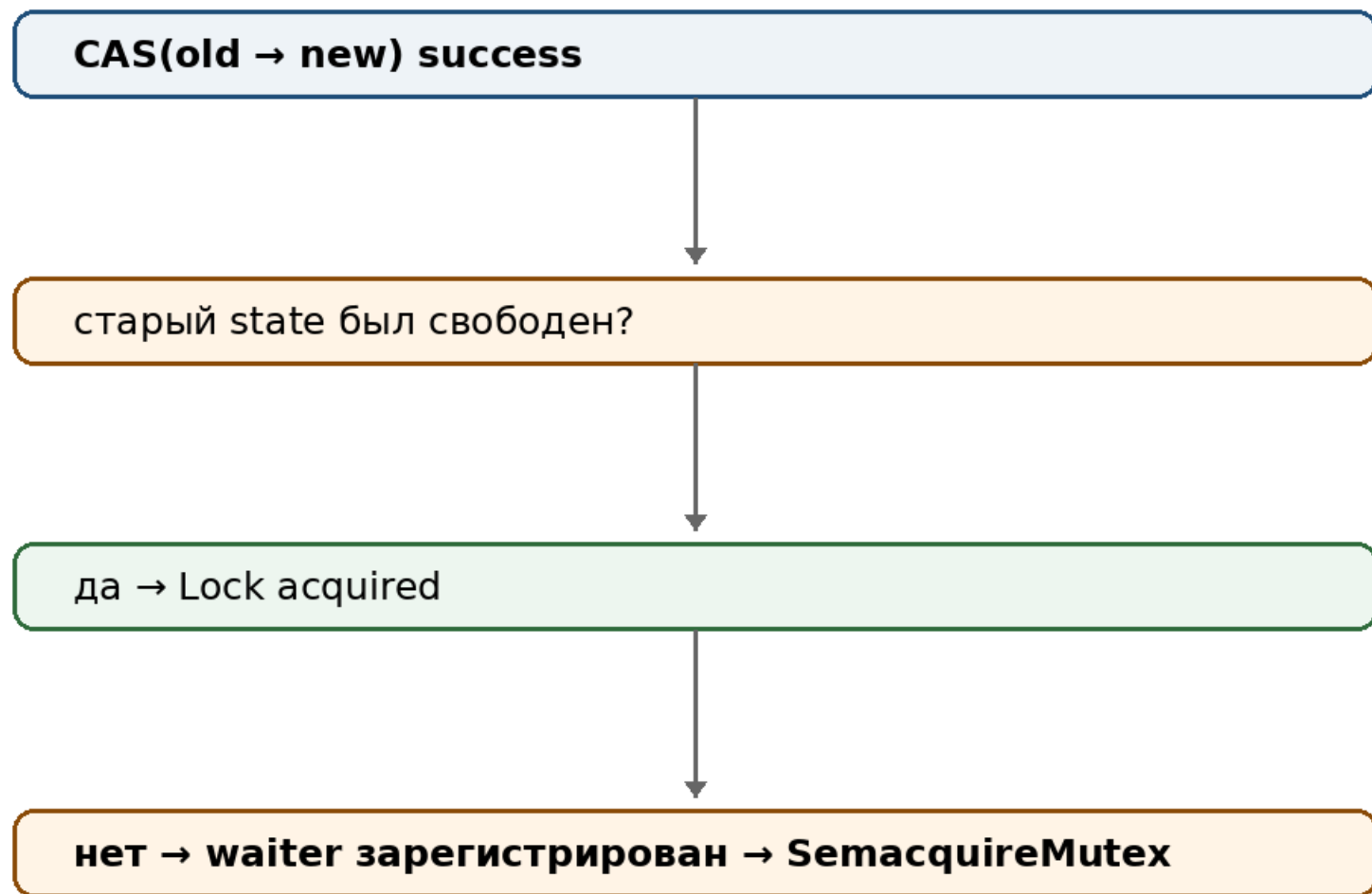
Normal

- Unlock будит waiter
- waiter снова конкурирует
- newcomer может украсть lock
- лучше throughput

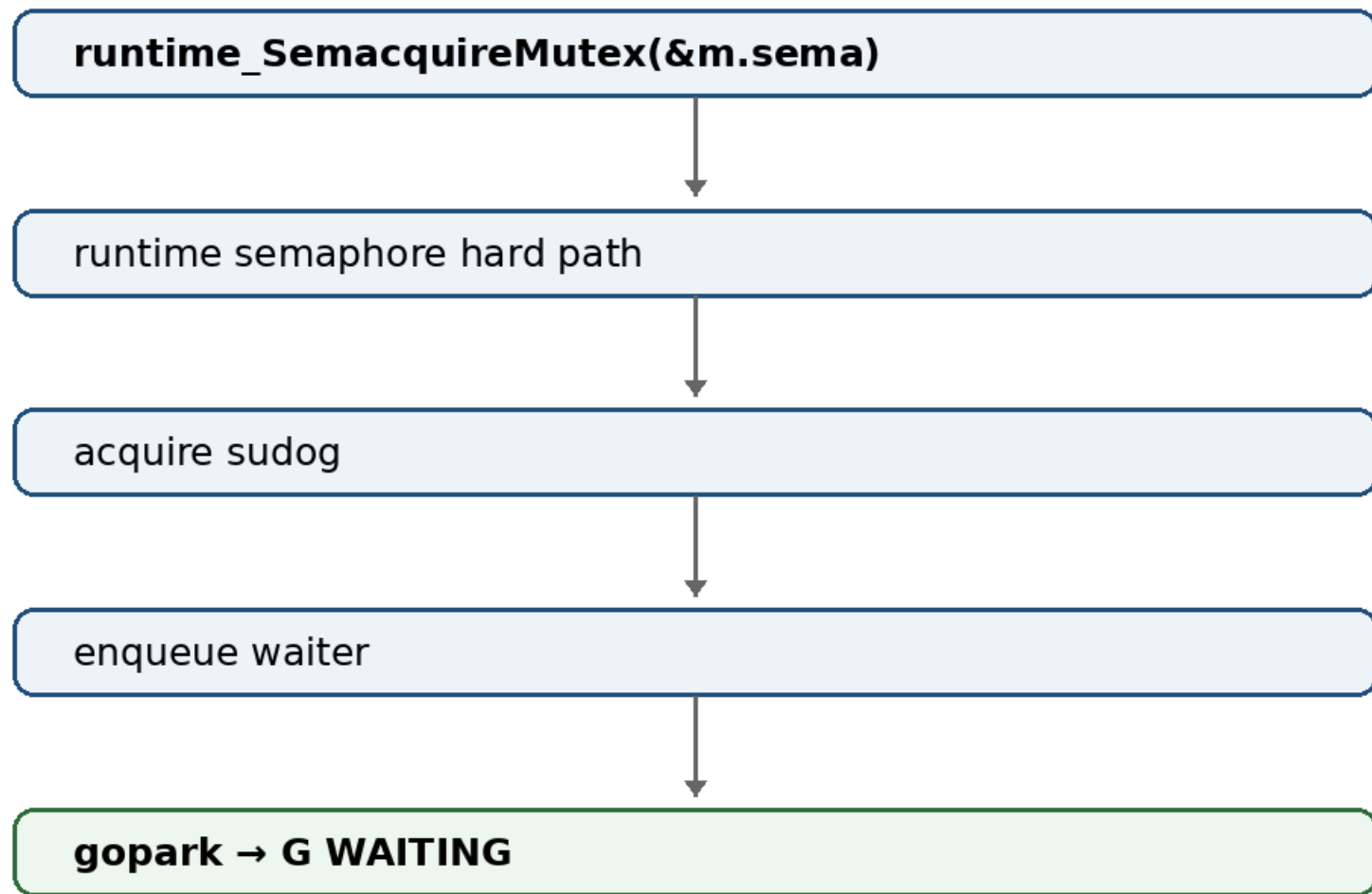
Starvation

- очередь получает приоритет
- newcomers не spin и не steal
- ownership передаётся следующему waiter
- лучше fairness / tail latency

CAS завершился успешно: два сценария



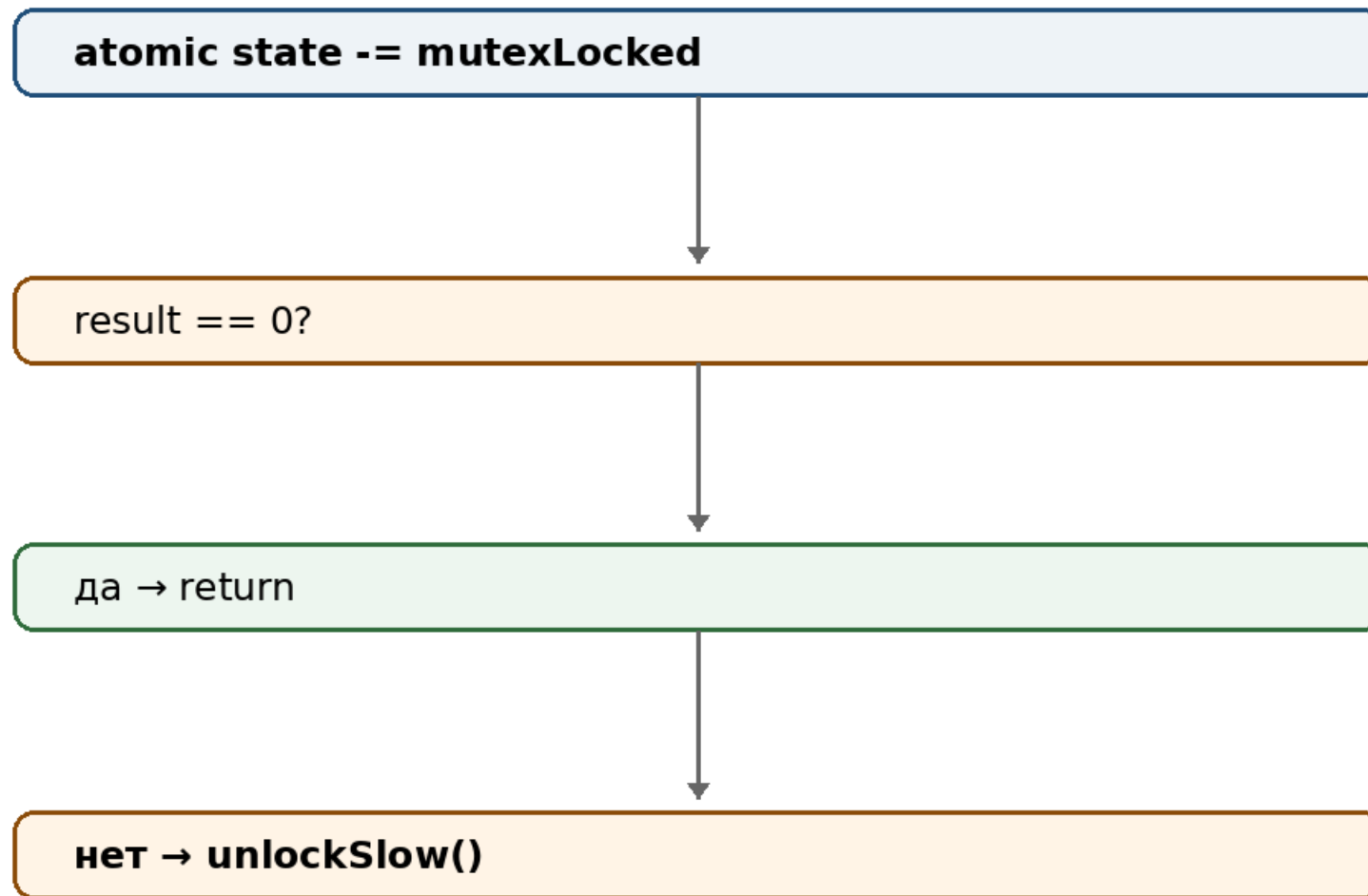
Slow waiting path



Что происходит с M/P во время ожидания

- Ожидающая G переходит RUNNING → WAITING.
- M больше не обязан быть связан с этой G.
- Scheduler может выполнить на том же P другую runnable goroutine.
- Поэтому contended Go Mutex обычно блокирует goroutine, а не выделенный OS thread.

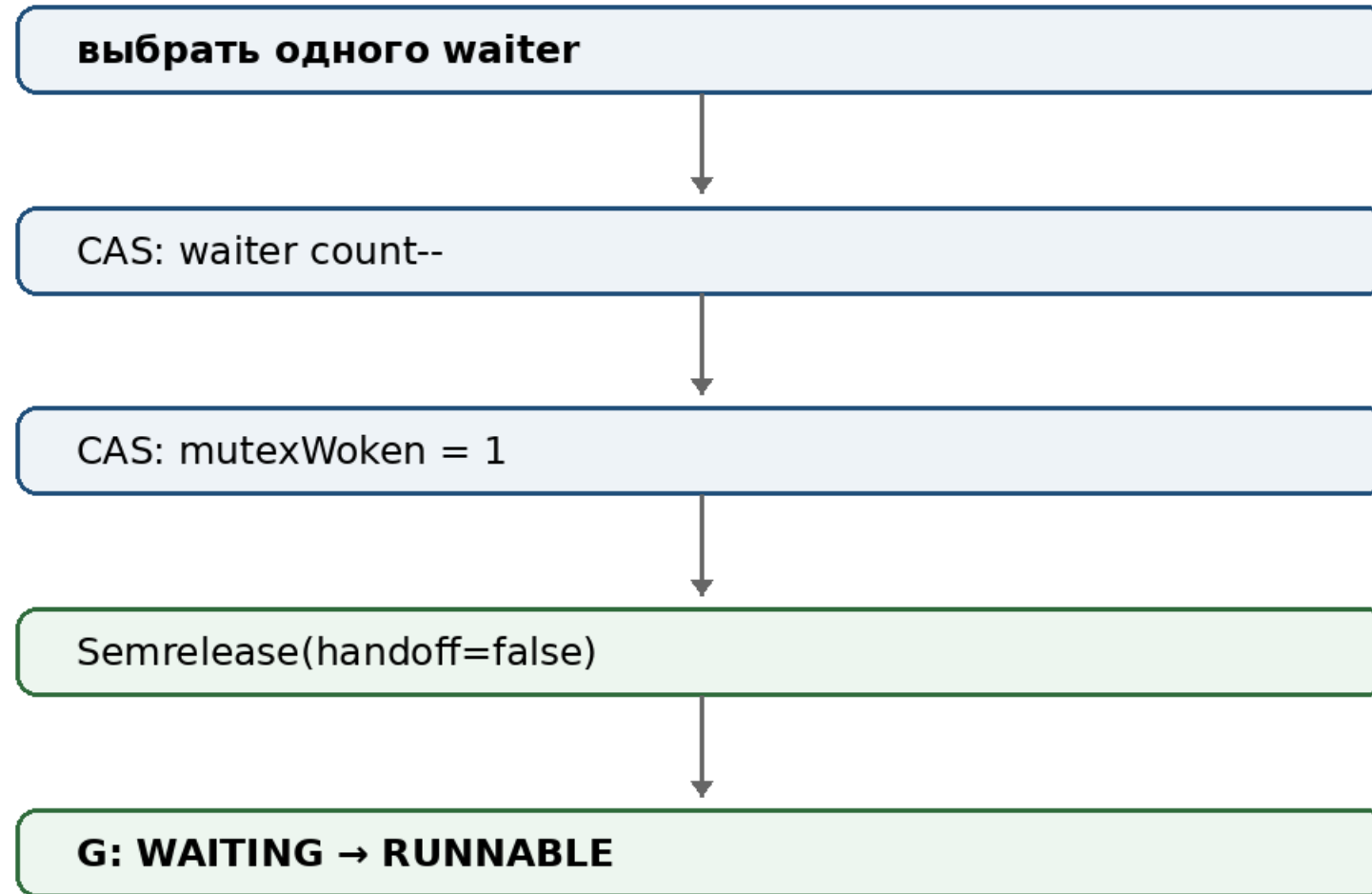
Unlock(): fast path тоже очень короткий



Normal unlockSlow: когда никого не будим

- `waiters == 0` — будить некого.
- `mutex` уже снова `Locked` — другой contender успел захватить его.
- `mutexWoken == 1` — активный candidate уже существует.
- `Starving == 1` — работает другой handoff protocol.

Normal wakeup protocol



Сначала фиксируем state, потом фактически будим G.

Starvation handoff

Unlock видит mutexStarving

Semrelease(handoff=true)

следующий waiter получает приоритет

awakened G корректирует state

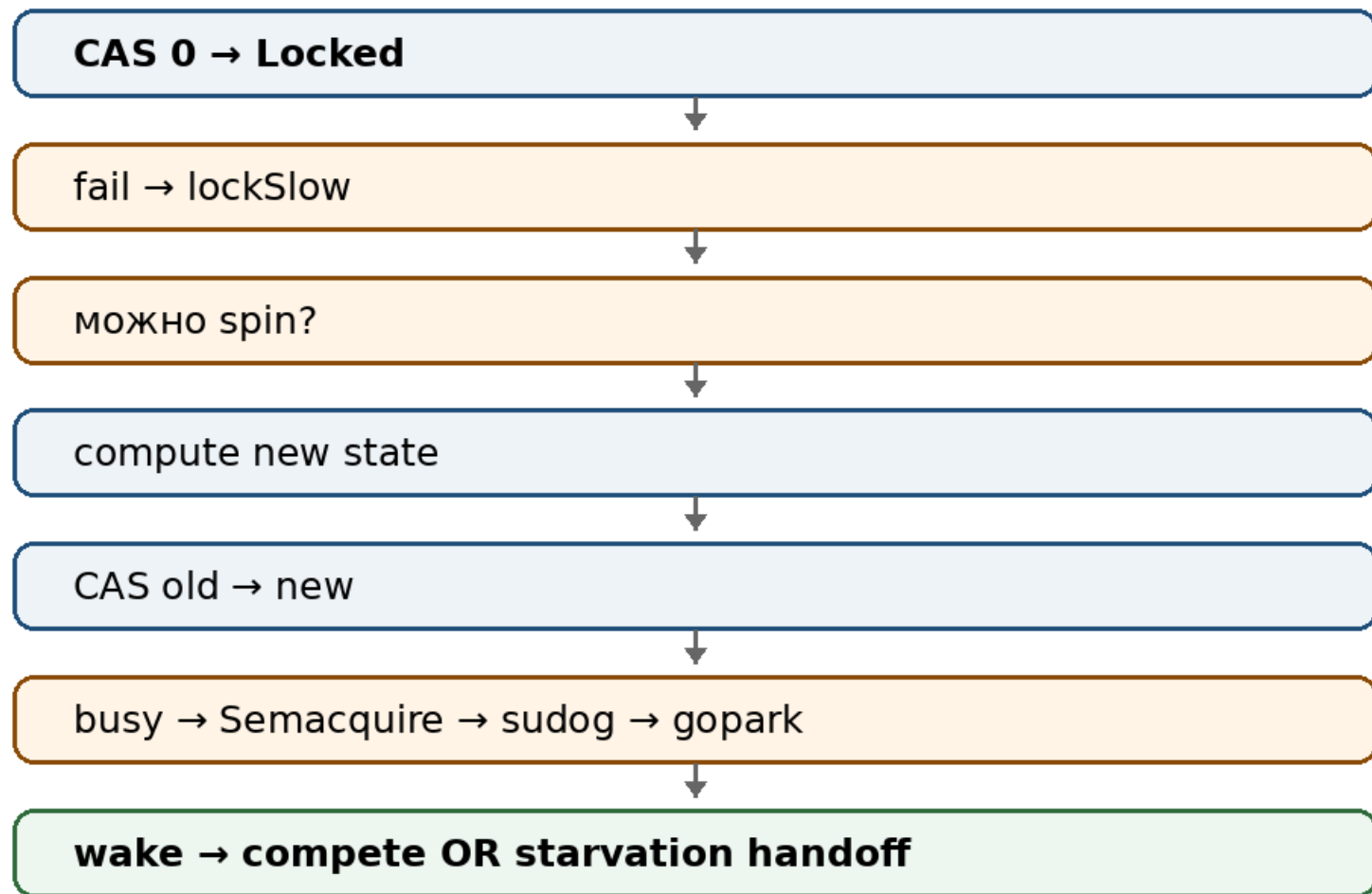
Locked=1; waiter--; возможно снять Starving

Почему в starvation встречается Locked = 0

```
state:
    mutexStarving = 1
    mutexLocked   = 0
    waiters       > 0
```

Locked=0 здесь не означает свободный Mutex: starvation bit резервирует ownership за очередью.

Полный Lock pipeline



Что меняется под нагрузкой

Low contention

- почти все Lock = один CAS
- минимум scheduler activity
- Mutex выглядит «бесплатным»

High contention

- CAS failures
- active spinning
- waiter queues
- park/wake
- serialization и p99 latency

Backend-антипаттерн: держать Mutex вокруг I/O

- `Lock → DB/HTTP/disk/sleep → Unlock` превращает один медленный downstream в очередь всех contenders.
- Симптом: CPU может быть невысоким, а latency огромной.
- Причина — serialization bottleneck и park/wake churn, а не «медленный CAS».

Диагностика

- Mutex profile — где время теряется на contended locks.
- Block profile — где goroutines реально блокируются.
- go tool trace — RUNNING → WAITING(sync.Mutex) → RUNNABLE → RUNNING.
- CPU profile — отделяет spinning/полезную работу от ожидания.
- Главный вопрос: сколько времени lock держат и что делают внутри critical section?

Что важно запомнить

- ``state`` — packed atomic state machine.
- Fast path = CAS; slow path = адаптивный протокол.
- ``mutexWoken`` ограничивает лишние wakeups.
- waiter count регистрируется до park — защита от lost wakeup.
- spinning меняет CPU на меньшую park/unpark latency.
- normal mode оптимизирует throughput; starvation mode — fairness.
- Под нагрузкой цена Mutex — очередь и serialization, а не сам atomic instruction.